

Sistemas Distribuídos e Computação em Nuvem

Aula 5 — Serviços web: REST e OpenAPI com FastAPI

C1 · Fundamentos e comunicação distribuída · 10/09/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — a trilha do semestre

C1 · Fundamentos e comunicação (Aulas 1–7)

Sockets, concorrência, gRPC, **REST** e IA como serviço.

C2 · Coordenação e consistência (Aulas 8–12)

Mensageria, relógios lógicos, **CAP**, **Raft**, resiliência.

C3 · Nuvem, implantação e segurança (Aulas 13–18)

Containers, nuvem, serverless, observabilidade, segurança.

 Você está na **Aula 5** — a interface **para fora** do seu serviço.

Retomada — o que você fez em casa

 A aula começa consolidando a entrega da Aula 4.

- Você **ampliou o .proto** com um segundo método e **regerou os stubs**?
- O **README** explicou **o impacto de mudar o contrato** (os dois lados regeram)?
- Ficou claro: **gRPC é a interface para dentro** (serviço  serviço)?

 Hoje montamos a interface **para fora** — a que o mundo consome: uma **API REST**, com documentação que **se escreve sozinha**.

Objetivos desta aula

Ao final, você será capaz de:

1. **Explicar** os princípios **REST** e a diferença para o **SOAP**.
2. **Construir** uma API REST com **FastAPI**, usando os **verbos** e **códigos** corretos.
3. **Ler e usar** a documentação automática gerada pelo **OpenAPI**.

Conceito 1/3 — Do SOAP ao REST

- **Serviços web** deixam sistemas diferentes conversarem pela internet.
- A geração anterior, **SOAP**, usava mensagens em **XML** e contrato rígido em **WSDL**: poderoso, mas **verboso e pesado** — sobrevive em legados e bancos.
- O **REST** (*Representational State Transfer*), dominante hoje, é **bem mais simples**: usa o **próprio HTTP como ele já é**, sem camada extra.

💡 Um serviço novo nasce **REST** — ou **gRPC** — quase nunca SOAP. Saber os dois ajuda a entender a evolução da **SOA** (Arquitetura Orientada a Serviços) rumo aos **microserviços**.

Conceito 2/3 — Recursos, verbos e status

O REST organiza tudo em torno de **recursos** (substantivos → URLs): `/tarefas` (a coleção), `/tarefas/7` (um item).

O verbo diz a ação

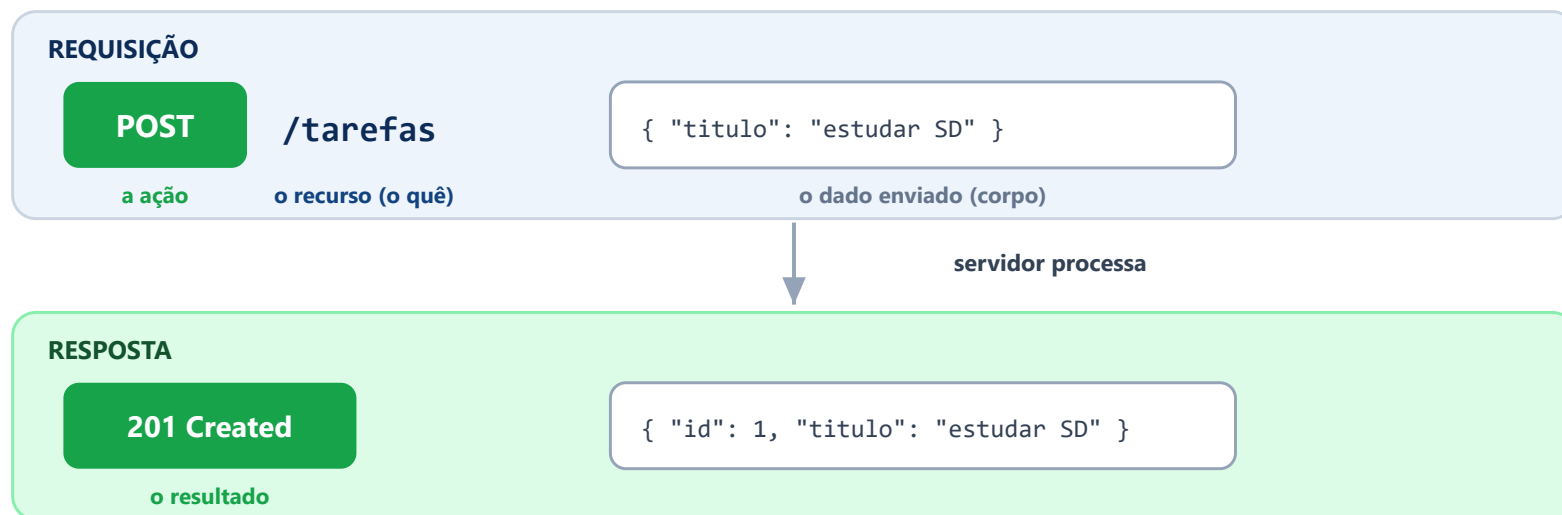
- **GET** — lê
- **POST** — cria
- **PUT** — atualiza
- **DELETE** — remove

O status diz o resultado

- **200** OK · **201** Criado
- **400** erro do cliente
- **404** não encontrado
- **500** erro do servidor

💡 A **URL** diz o *quê*, o **verbo** diz a *ação*, o **status** diz o *resultado*. Essa separação é o que torna a API previsível.

Anatomia de uma requisição REST



💡 **Em miúdos:** é como um balcão dos Correios: o **verbo** é o que você quer fazer (enviar, consultar), a **URL** é o balcão certo, o **status** é o carimbo de volta (deu certo · não achei · deu erro).

Conceito 2/3 — Idempotência (guarde para depois)

- Uma operação é **idempotente** quando, **repetida**, leva ao **mesmo estado final**.
- **GET, PUT e DELETE** são idempotentes; **POST não é** — cada chamada **cria um novo** recurso.

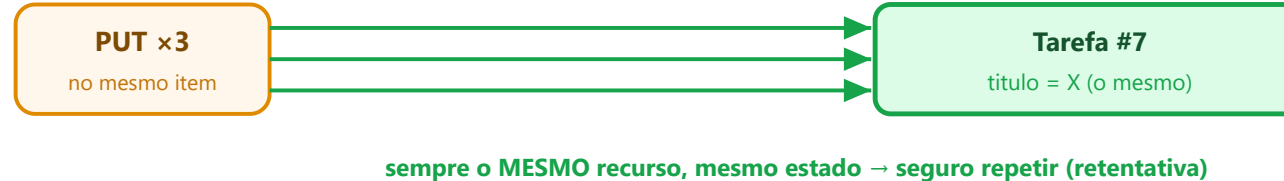
⚠ Essa distinção volta a importar na **resiliência** (Aula 11): só é seguro **repetir automaticamente** (retentativa) uma operação **idempotente**. Repetir um POST pode criar recursos duplicados — a semente da **idempotência** que você viu na Aula 1.

Idempotência, visualmente

POST /tarefas — repetido 3× · NÃO idempotente



PUT /tarefas/7 — repetido 3× · idempotente



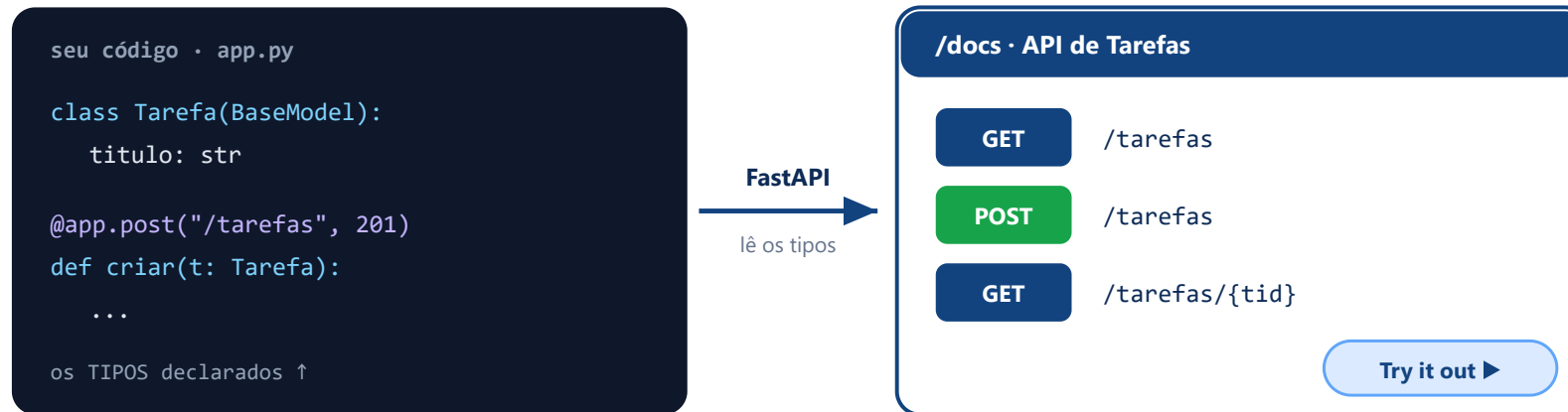
💡 **Em miúdos:** apertar o botão de **chamar o elevador** 3× não chama 3 elevadores (idempotente); **enviar um formulário** 3× cria 3 pedidos (não idempotente). Por isso só se pode **repetir sozinho** o que é idempotente.

Conceito 3/3 — OpenAPI: a documentação que não envelhece

- Uma boa API REST precisa de um **contrato** para quem vai consumi-la.
- Com o **FastAPI**, esse contrato — no padrão **OpenAPI** — é **gerado automaticamente** a partir dos **tipos declarados no código**, numa **página interativa** (`/docs`).
- Como **nasce do próprio código**, a documentação **não desatualiza** — o problema crônico da era SOAP, em que contrato e implementação viviam divergindo.

💡 Você escreve os tipos uma vez; o FastAPI valida a entrada, gera o `/docs` e mantém tudo em sincronia.

A documentação nasce do código



you didn't write documentation — it was born from the **types**; change the code → change the /docs

💡 **Em miúdos:** a bula já sai **impressa do próprio remédio**. Como o /docs vem dos tipos do código, ele **nunca fica desatualizado** — o velho problema do WSDL/SOAP, em que a doc vivia divergindo.

Laboratório

Uma API REST com FastAPI — passo a passo

Lab · Passo 1 — instalar e o esqueleto (`app.py`)

```
pip install fastapi "uvicorn[standard]"
```

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="API de Tarefas")
tarefas = []                                # "banco" em memória (só para a aula)

class Tarefa(BaseModel):                    # o TIPO da entrada → vira validação + /docs
    titulo: str
```

💡 O `BaseModel` do Pydantic declara o formato do dado. É dele que o FastAPI tira a **validação** e a **documentação**.

Lab · Passo 2 — as 3 rotas

```
@app.get("/tarefas")                # LER a coleção
def listar():
    return tarefas

@app.post("/tarefas", status_code=201)  # CRIAR (201 = Created)
def criar(tarefa: Tarefa):
    nova = {"id": len(tarefas) + 1, "titulo": tarefa.titulo}
    tarefas.append(nova)
    return nova

@app.get("/tarefas/{tid}")            # LER um item
def obter(tid: int):
    for t in tarefas:
        if t["id"] == tid:
            return t
    raise HTTPException(status_code=404, detail="tarefa não encontrada")
```

Lab · Passo 3 — rodar e abrir o `/docs`

```
uvicorn app:app --reload --port 8000  
# abra no navegador: http://localhost:8000/docs
```

- No `/docs` (OpenAPI) você **testa as rotas pelo navegador**: clique em **POST `/tarefas`**, *Try it out*, envie `{"titulo": "estudar SD"}` → resposta **201**.
- Depois **GET `/tarefas`** lista o que criou; **GET `/tarefas/99`** devolve **404**.

💡 Você não escreveu **uma linha** de documentação — o `/docs` nasceu dos **tipos** do seu código.

Lab · Checkpoints & problemas comuns (Windows)

✓ O que você deve ver

- `uvicorn` sobe em `http://127.0.0.1:8000`.
- `/docs` abre a página interativa.
- POST devolve **201**; GET de id inexistente devolve **404**.

🔧 Se der erro

- No module named fastapi → `pip install fastapi "uvicorn[standard]"`.
- `app:app = arquivo app.py : variável app`.
Nome diferente? Ajuste.
- Porta ocupada → troque `8000`.
- Alterou o código? O `--reload` recarrega sozinho.

No seu trabalho — C1.A2

- A **outra interface** do trabalho é **REST**. As rotas de **submissão** e de **consulta do resultado** são o **coração** da API.

💡 Puxa direto para o kit:

- **TAREFA 1** — POST /predict que **enfileira** a tarefa e devolve um **id**.
- **TAREFA 2** — GET /resultado/{id} para buscar o resultado quando pronto.
- **TAREFA 6** — registrar **log** de cada requisição.

Repositório: sd-2026-2-kit-c1a2 (a rota síncrona já roda; veja a Aula 1).

Atividade para casa — completar o CRUD

1. **Adicione** `PUT /tarefas/{id}` (atualiza o título) e `DELETE /tarefas/{id}` (remove) — com os **status** corretos.
2. **Trate os erros:** id inexistente → **404**.
3. **Escreva 3 testes** (use `TestClient` do FastAPI): criar (201), listar (200), buscar id inexistente (404).
4. Confirme no `/docs` que as novas rotas aparecem **sozinhas**.

 **Modelo pronto:** `exemplos/aula05/test_api_EXEMPLO.py` tem o **1º teste feito** + 2 `# TODO`. Instale `pip install pytest httpx` e rode `pytest -v`.

 **Entregar até a próxima aula:** API REST com **CRUD completo** e **3 testes passando**.

◆ Foco ENADE

O que costuma cair:

- **Web services: SOAP × REST** — diferenças e aplicabilidade.
- **Verbos HTTP, códigos de status e idempotência.**
- **SOA** (Arquitetura Orientada a Serviços) e **interoperabilidade.**
- **Contratos de API** e documentação (**WSDL, OpenAPI**).

Termos-chave: REST · SOAP · Recurso · Verbo HTTP · Código de status · OpenAPI · Idempotência

💡 Decore o mapa: **POST→201**, **GET→200**, **404** não encontrado, **400** cliente errou, **500** servidor falhou. E qual verbo é idempotente.

Questão de autoavaliação (estilo ENADE)

Em uma API REST, para **criar** um novo recurso e sinalizar **sucesso**, o verbo e o código de status esperados são, respectivamente:

- A) **GET** e **200** (OK).
- B) **POST** e **201** (Created).
- C) **PUT** e **204** (No Content).
- D) **POST** e **404** (Not Found).
- E) **DELETE** e **200** (OK).

Resolução — alternativa B

- **POST** cria o recurso e **201 (Created)** indica **criação bem-sucedida**.
- **D** erra o status: **404** sinaliza recurso **inexistente** — não faz sentido como resposta de criação.
- **A**, **C** e **E** usam verbos que **não criam** (ler, atualizar, remover).

💡 É a rota `POST /tarefas` do seu lab de hoje virando questão de prova.

Fora da sala · Glossário

Para estudar

- **Coulouris**, cap. 9 — Serviços web.
- Documentação oficial do **FastAPI** (o tutorial é excelente).
- Abra o `app/api_rest.py` do **kit da C1** e compare com o seu.

Glossário

- **REST**: recursos em URLs + verbos HTTP.
- **SOAP**: web service em XML, contrato em WSDL.
- **Recurso**: entidade exposta por uma URL.
- **Código de status**: número que informa o resultado.
- **Idempotência**: repetir não muda o resultado final.
- **OpenAPI**: contrato da API gerado pelo FastAPI.

Até a próxima aula

Entregue a API com CRUD completo + 3 testes. A Aula 6 (EAD) consolida o serviço.

← Anterior
Aula 4 · gRPC

≡ Índice
todas as aulas

Próxima aula →
Aula 6 · IA como serviço (EAD)